

Research Proposal

Proposed Title

Adaptive Cost-, Reliability-, and Checkpoint-Aware Cloud GPU Orchestration for Persistent Machine Learning Training

Alternative shorter title

Adaptive Cost-Aware Cloud GPU Orchestration for Persistent ML Training

1. Introduction

Machine learning training increasingly depends on GPU computing resources, but access to suitable GPUs can be expensive and unpredictable, particularly when workloads run for many hours or days. Cloud GPU platforms provide access to heterogeneous GPU resources with different prices, performance characteristics, availability levels and interruption risks.

For long-running machine learning workloads, selecting a GPU solely according to its hourly price or computational performance may result in inefficient resource utilization. A low-cost GPU may have a higher probability of interruption, while a high-performance GPU may complete a workload faster but incur a substantially higher total cost. Similarly, selecting a resource without considering the current training progress and checkpoint state can increase the amount of computation that must be repeated following a failure.

Existing GPU scheduling research has investigated several important dimensions including GPU utilization, job completion time, fairness, resource sharing, heterogeneous GPU selection and adaptive resource allocation. However, these dimensions are often investigated independently or within large GPU clusters where the primary concern is cluster-wide scheduling efficiency.

This research proposes an adaptive cloud GPU orchestration approach for persistent machine learning training that considers multiple characteristics of a training job and available GPU resources simultaneously. The proposed approach will dynamically evaluate available GPU resources using factors such as cost, expected performance, reliability, interruption risk, training progress and checkpoint recovery state.

The proposed system will be implemented as part of a cloud notebook and persistent training platform. The platform will provide an experimental environment in which long-running machine learning jobs can be submitted, monitored, checkpointed, recovered and migrated between cloud GPU resources.

The existing project architecture already separates the browser-facing control plane from the compute plane, allowing a submitted training job to continue after the user's browser is closed.

The research will focus primarily on the orchestration and resource-selection problem rather than the notebook interface itself.

2. Background

2.1 Cloud GPU Computing

Modern machine learning workloads frequently require GPU acceleration. Cloud GPU providers allow users to rent GPU resources without owning physical hardware.

However, cloud GPU environments are heterogeneous.

Different GPU instances may differ in:

- GPU model
- GPU memory
- computational throughput
- hourly price
- availability
- geographic location
- reliability
- interruption/preemption probability
- startup time

Therefore, selecting an appropriate GPU is a multi-dimensional decision rather than a simple price comparison.

2.2 Persistent Machine Learning Training

Interactive notebook environments are useful for experimentation but are not necessarily designed for long-running training jobs.

The proposed platform treats training as a durable background job rather than something tied to a browser session.

The project's current design specifies that the browser submits a job and receives a job identifier while the actual training executes independently in a remote container.

This creates an appropriate environment for studying long-running GPU orchestration.

2.3 GPU Scheduling

Previous research has demonstrated that GPU scheduling can significantly improve cluster performance.

Gandiva uses characteristics of deep-learning workloads to dynamically time-share and migrate jobs between GPUs. Its evaluation demonstrated improved GPU utilization and faster hyperparameter exploration. [1]

Tiresias investigates GPU cluster scheduling with the objective of reducing deep-learning job completion time. [2]

AntMan introduces dynamic resource scaling and GPU sharing for large-scale production workloads. [3]

Pollux extends scheduling by jointly considering resource allocation and the training configuration of individual jobs. [4]

Sia addresses heterogeneous GPU clusters and dynamically matches jobs to different GPU types and resource configurations. [5]

These studies demonstrate that adaptive GPU resource management is a significant research area.

3. Problem Statement

Cloud GPU users currently face a resource-selection problem when executing long-running machine learning workloads.

A GPU that appears cheapest at the beginning of a training job may not produce the lowest overall training cost if:

- it has significantly lower performance;
- the workload is interrupted;
- the GPU becomes unavailable;
- the training process must restart;
- a checkpoint is too old;
- recovery requires significant recomputation.

Conversely, selecting the most powerful GPU may unnecessarily increase cost for workloads that can tolerate slower execution.

Therefore, there is a need for an orchestration mechanism that evaluates GPU resources according to the **overall expected cost and completion behavior of a training job**, rather than considering only one factor such as hourly price or raw performance.

The research problem can therefore be formulated as:

How can cloud GPU resources be dynamically selected and managed for persistent machine learning training by jointly considering cost, performance, reliability, interruption risk and checkpoint recovery state?

4. Research Gap

The literature demonstrates substantial research in GPU scheduling.

However, the following areas provide an opportunity for investigation:

Existing research has studied:

1. GPU cluster utilization
2. Job completion time
3. Fairness
4. GPU sharing
5. Heterogeneous GPU scheduling
6. Adaptive resource allocation
7. Goodput-aware scheduling
8. Preemptible-resource checkpointing
9. Cost-efficient training

For example, Pollux jointly optimizes job configuration and cluster resource allocation, while Sia focuses on heterogeneous GPU types and adaptive resource configurations. [4][5]

The 2026 work by Desai et al. specifically investigates prediction-informed checkpointing and hybrid use of spot and on-demand resources. [6]

Furthermore, CloudX demonstrates that a Jupyter-based cloud architecture can combine notebook execution with GPU-aware resource scheduling. [7]

Proposed research gap

The proposed research will investigate the combination of:

cloud GPU cost + expected performance + reliability + interruption risk + training progress + checkpoint recovery state

within a persistent training orchestration environment.

The central hypothesis is that **resource selection should depend not only on the characteristics of the GPU, but also on the current state of the training job.**

For example:

A job at 5% progress with no checkpoint may have different optimal resource requirements from the same job at 95% progress with a recent checkpoint.

Similarly:

A cheap interruptible GPU may be appropriate when the expected recovery cost is low, but an on-demand GPU may become preferable when the training job is close to completion or when interruption probability increases.

This creates a dynamic decision problem that will form the central research contribution.

Important: this should be presented as a *proposed research gap to be validated through the systematic literature review*, rather than claiming that no previous system has ever considered these factors.

5. Aim

The aim of this research is:

To design, implement and evaluate an adaptive cloud GPU orchestration approach that dynamically selects and manages GPU resources for persistent machine learning training by jointly considering computational cost, performance, reliability, interruption risk and checkpoint recovery state.

6. Research Objectives

Objective 1 — Literature and Gap Analysis

Conduct a systematic review of existing research on:

- GPU scheduling
 - cloud GPU resource allocation
 - heterogeneous GPU scheduling
 - cost-aware scheduling
 - preemptible/spot GPU computing
 - checkpoint-aware training
 - failure recovery
 - persistent machine learning infrastructure
-

Objective 2 — Develop a GPU Resource Evaluation Model

Develop a model for evaluating available GPU resources according to factors such as:

- hourly cost
 - estimated training throughput
 - GPU memory
 - expected reliability
 - interruption probability
 - startup time
 - current training progress
 - checkpoint age
 - expected recovery cost
-

Objective 3 — Develop an Adaptive GPU Selection Algorithm

Design an algorithm that dynamically determines the most appropriate GPU resource for a running or queued training job.

The algorithm should be capable of:

- selecting an initial GPU;
 - reconsidering the selected resource;
 - responding to changes in GPU availability;
 - responding to changes in interruption risk;
 - considering training progress;
 - using checkpoint information when making migration/recovery decisions.
-

Objective 4 — Implement the Experimental Platform

Develop a prototype cloud GPU training platform that provides:

- notebook environment;
- job submission;
- persistent background training;
- GPU provisioning;
- checkpoint storage;
- job monitoring;
- failure detection;
- recovery;
- GPU release;
- cost tracking.

The proposed project's existing architecture already provides a strong foundation for this experimental platform using Next.js, FastAPI, PostgreSQL, Redis, JupyterHub/JupyterLab, Docker, cloud GPU infrastructure and object storage.

Objective 5 — Evaluate the Proposed Approach

Compare the proposed approach against multiple baseline strategies using controlled experiments.

The evaluation will measure:

- total training cost;
 - job completion time;
 - GPU utilization;
 - recovery time;
 - computation lost after failures;
 - number of interruptions;
 - checkpoint overhead;
 - successful completion rate;
 - cost per successfully completed job.
-

7. Research Questions

Primary Research Question

RQ1:

How can cloud GPU resources be dynamically orchestrated for persistent machine learning training to minimize overall training cost while maintaining acceptable training performance and reliability?

Secondary Questions

RQ2:

How does incorporating GPU performance, cost and reliability into resource selection affect total training cost and job completion time?

RQ3:

How does checkpoint-aware resource selection affect recovery time and computational loss after GPU interruptions?

RQ4:

Can adaptive GPU selection outperform static strategies such as cheapest-GPU and fastest-GPU selection?

RQ5:

How does the proposed approach perform under different interruption probabilities, GPU price differences and workload characteristics?

8. Research Hypotheses

H1

An adaptive multi-factor GPU selection strategy will achieve lower total training cost than selecting the cheapest available GPU.

H2

An adaptive strategy will achieve a better cost-performance trade-off than selecting the fastest available GPU.

H3

Checkpoint-aware recovery will reduce computational work lost after GPU interruptions compared with restarting training from the beginning.

H4

Considering interruption probability and training progress will improve successful job completion rates when compared with static GPU-selection strategies.

H5

The proposed approach will provide a better overall cost-reliability-performance trade-off than single-factor resource selection strategies.

9. Scope

In Scope

The research will investigate:

- cloud GPU training;
- long-running machine learning jobs;
- heterogeneous GPU resources;
- GPU cost;
- GPU performance;
- reliability;
- interruption/preemption;
- checkpointing;
- failure recovery;
- adaptive GPU selection;
- persistent training jobs;
- cost-aware orchestration.

Out of Scope

The research will not primarily focus on:

- developing a new neural network architecture;
- improving model accuracy;
- designing a new GPU;
- GPU hardware architecture;
- large-scale distributed training algorithms;
- general-purpose cloud security;
- payment processing;
- commercial billing;
- production-scale public cloud infrastructure.

The notebook interface is primarily an experimental interface rather than the research contribution.

10. Proposed System Architecture

The system will contain two major planes.

10.1 Control Plane

Responsible for:

- user authentication;
- notebook management;
- job submission;
- job state;
- GPU resource evaluation;
- scheduling;
- quota management;
- cost estimation;
- monitoring;
- notifications.

10.2 Compute Plane

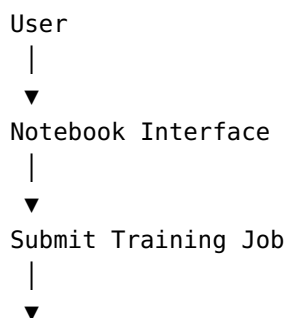
Responsible for:

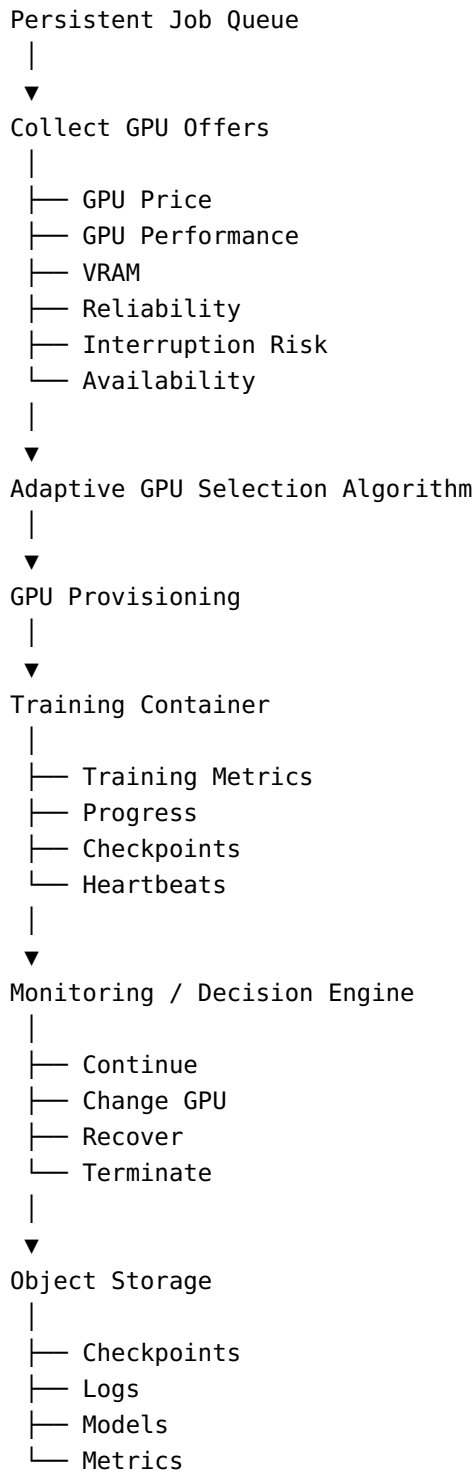
- notebook execution;
- model training;
- GPU execution;
- checkpoint creation;
- artifact generation;
- metrics collection;
- recovery.

The existing project report already defines this control-plane/compute-plane separation.

11. Proposed Workflow

The proposed workflow is:





The current implementation plan already proposes storing checkpoints and metrics in object storage and recording Docker image, dataset version, hyperparameters and Git commit for reproducibility.

12. Proposed Adaptive Orchestration Model

The central component of the research will be the **Adaptive GPU Decision Engine**.

For every candidate GPU resource, the system will estimate its expected suitability for the current training job.

Potential factors include:

Cost

Expected GPU rental cost.

Performance

Expected training throughput on the selected GPU.

Reliability

Expected probability that the resource remains available throughout the required period.

Interruption Risk

Probability of interruption or preemption.

Recovery Cost

Expected amount of computation that could be lost if the resource fails.

Training Progress

Current percentage/iteration/epoch completed.

Checkpoint State

Age and validity of the latest checkpoint.

13. Initial Mathematical Formulation

A possible initial formulation is:

$$Score(g, j, t) = w_c C(g, j, t) + w_p P(g, j, t) + w_r R(g, j, t) + w_i I(g, j, t) + w_k K(g, j, t)$$

where:

- g = candidate GPU
- j = training job
- t = current decision time
- C = normalized cost
- P = normalized performance penalty
- R = reliability penalty
- I = interruption-risk penalty

- K = expected checkpoint/recovery penalty
- w_c, w_p, w_r, w_i, w_k = weighting parameters.

The system selects:

$$g^* = \arg \min_{g \in G} \text{Score}(g, j, t)$$

However, this formulation should **not be presented as the final algorithm yet**.

The final mathematical formulation should be determined after the literature review identifies which objective functions, constraints and optimization strategies are already established.

That is important for avoiding the weak research argument of simply inventing a weighted score.

14. Dynamic Decision-Making

The important difference between the proposed approach and static GPU selection is that the decision can be repeated during training.

For example:

At 5% training progress

The system may choose:

Cheap interruptible GPU

because the potential recovery cost is relatively small.

At 50%

The decision may depend on:

interruption probability + checkpoint age + remaining training time.

At 95%

The system may prefer:

reliable on-demand GPU

because losing the final stage of training could be more expensive than paying a higher hourly price.

This creates the central research concept:

The optimal GPU resource may change as the training job evolves.

15. Checkpoint-Aware Recovery

The system will periodically save:

- model state;
- optimizer state;
- training epoch/iteration;
- metrics;
- configuration;
- dataset version;
- environment information.

The existing project design already proposes checkpointing every epoch or fixed interval and resuming from the newest valid checkpoint after failure.

The research will extend this from a simple reliability mechanism into a **decision variable**.

For example:

$$ExpectedRecoveryCost = P(interruption) \times ExpectedLostTraining \times CostPerTrainingTime$$

This allows the scheduler to consider whether a cheap but risky GPU is actually economical.

16. Baseline Methods

A strong experimental evaluation should not compare the proposed method only against a random strategy.

At minimum, use:

Baseline 1 — Cheapest GPU

Always select the GPU with the lowest hourly price.

Baseline 2 — Fastest GPU

Always select the GPU with the highest expected throughput.

Baseline 3 — Random GPU

Select a feasible GPU randomly.

Baseline 4 — Cost/Performance Heuristic

Select the GPU using a simple cost-per-performance ratio.

Baseline 5 — Static Reliability-Aware Strategy

Select the GPU using cost + reliability, but without dynamic training-state information.

Proposed Method

Adaptive strategy considering:

- cost;
- performance;
- reliability;
- interruption risk;
- training progress;
- checkpoint state.

This is a much stronger experimental design.

17. Experimental Design

The experiment should simulate or execute multiple training workloads under different cloud GPU conditions.

Experiment A — GPU Price Variation

Evaluate the system when GPU prices differ significantly.

Measure:

- total cost;
 - completion time;
 - GPU selection frequency.
-

Experiment B — Performance Variation

Use GPUs with different computational capabilities.

Measure:

- training throughput;
 - job completion time;
 - cost per completed job.
-

Experiment C — Interruption Variation

Introduce different interruption probabilities.

Example levels:

- low interruption;
- medium interruption;
- high interruption.

Measure:

- failed jobs;
 - recovery time;
 - computation lost;
 - total cost.
-

Experiment D — Checkpoint Interval

Compare:

- no checkpoint;
- fixed checkpoint interval;
- adaptive checkpoint interval.

Measure:

- checkpoint overhead;
- recovery time;
- lost computation;
- total training cost.

The 2026 preemptible-VM study provides especially strong motivation for this experiment because it found substantial benefits from dynamically tuning checkpoint intervals instead of using fixed intervals.

Experiment E — Training Progress

Trigger resource decisions at different stages:

- 10%;
- 25%;
- 50%;
- 75%;
- 90%.

Investigate whether the optimal resource choice changes as the job approaches completion.

Experiment F — Multi-User Workload

Submit multiple jobs simultaneously.

Evaluate:

- average cost;
- queueing time;
- fairness;
- GPU utilization;
- JCT;
- resource allocation.

This connects the proposed system with the broader GPU scheduling literature.

18. Evaluation Metrics

Cost Metrics

Total GPU Cost

$$TotalCost = \sum GPU\ Hour \times GPU\ Price$$

Cost per Completed Job

$$CostPerJob = \frac{TotalCost}{Number\ of\ Successfully\ Completed\ Jobs}$$

Performance Metrics

Job Completion Time

Time from job submission to successful completion.

Training Throughput

Examples:

- samples/sec;
- iterations/sec;
- epochs/hour.

GPU Utilization

Percentage of available GPU capacity being used.

Reliability Metrics

Failure Rate

$$FailureRate = \frac{FailedJobs}{TotalJobs}$$

Recovery Time

Time required to resume training after failure.

Lost Computation

Training work performed after the latest checkpoint and before failure.

Checkpoint Metrics

- checkpoint frequency;
 - checkpoint storage overhead;
 - checkpoint creation time;
 - recovery time;
 - stale checkpoint duration.
-

Overall Metric

A particularly useful metric will be:

Cost per successfully completed training job

because a very cheap GPU that repeatedly fails may actually be more expensive than a reliable GPU.

19. Experimental Dataset / Workloads

The project does not necessarily require a conventional machine-learning dataset as its research dataset.

Instead, the experimental workload can consist of controlled training jobs.

Possible workloads:

Workload 1

Small CNN image classification.

Workload 2

Medium-size image classification.

Workload 3

Transformer fine-tuning.

Workload 4

Object detection.

Workload 5

Long-running synthetic training workload.

The purpose is not to achieve state-of-the-art model accuracy.

The purpose is to generate representative GPU training workloads with different:

- memory requirements;
 - execution durations;
 - computational requirements;
 - checkpoint sizes;
 - convergence characteristics.
-

20. Failure Injection

A major strength of this research could be controlled failure injection.

During experiments, intentionally terminate or interrupt the GPU training environment.

For example:

```
Training
  ↓
GPU interruption
  ↓
Detect failure
  ↓
Locate latest checkpoint
  ↓
Evaluate available GPUs
  ↓
Adaptive GPU selection
  ↓
Provision new GPU
```

↓
Resume training

This allows the research to measure whether adaptive orchestration actually reduces the impact of failures.

21. Comparison Framework

The experiment should ultimately produce comparisons such as:

Strategy	Cost	JCT	Recovery	Failure Rate	GPU Utilization
Cheapest					
Fastest					
Random					
Cost/Performance					
Proposed Adaptive					

The actual values will be obtained experimentally rather than predetermined.

22. Expected Results

The research expects that the proposed adaptive approach will:

1. reduce total training cost compared with always selecting high-performance GPUs;
2. reduce failure-related cost compared with always selecting the cheapest interruptible resources;
3. reduce computational loss through checkpoint-aware recovery;
4. improve successful job completion rate;
5. achieve a better cost-performance-reliability trade-off;
6. dynamically change GPU selection according to workload and training state.

These are **expected outcomes, not guaranteed findings**.

The experiments may demonstrate that the proposed method does not outperform certain baselines under some conditions. Such results would still be scientifically useful if the conditions are clearly analyzed.

23. Research Contribution

The expected contributions are:

Contribution 1 — Adaptive GPU Orchestration Model

A model for evaluating cloud GPU resources using multiple dimensions rather than a single cost or performance metric.

Contribution 2 — Checkpoint-Aware Resource Selection

Integration of training progress and checkpoint state into GPU resource decisions.

Contribution 3 — Cost-Reliability Optimization

A mechanism for balancing GPU rental cost against interruption and recovery risk.

Contribution 4 — Persistent ML Training Testbed

A practical cloud notebook/training platform that enables controlled evaluation of persistent GPU workloads.

Contribution 5 — Experimental Evaluation

A systematic comparison of adaptive resource selection against static and heuristic approaches.

24. Novelty Claim

The proposal should make a **careful** novelty claim.

Do not write:

“No previous research has investigated cloud GPU scheduling based on cost, performance and reliability.”

That would be too broad and is unlikely to survive peer review.

Instead write:

This research investigates an adaptive orchestration approach that jointly incorporates cloud GPU cost, expected performance, reliability, interruption risk, training progress and checkpoint recovery state when selecting and managing resources for persistent machine learning workloads.

The novelty is therefore in the **combination, decision process and evaluation methodology**, subject to confirmation through the final systematic literature review.

This distinction is particularly important because CloudX already combines interactive notebooks with GPU-aware scheduling, and Sia/Pollux already demonstrate sophisticated adaptive GPU scheduling.

25. Relationship to Existing CloudGPU Project

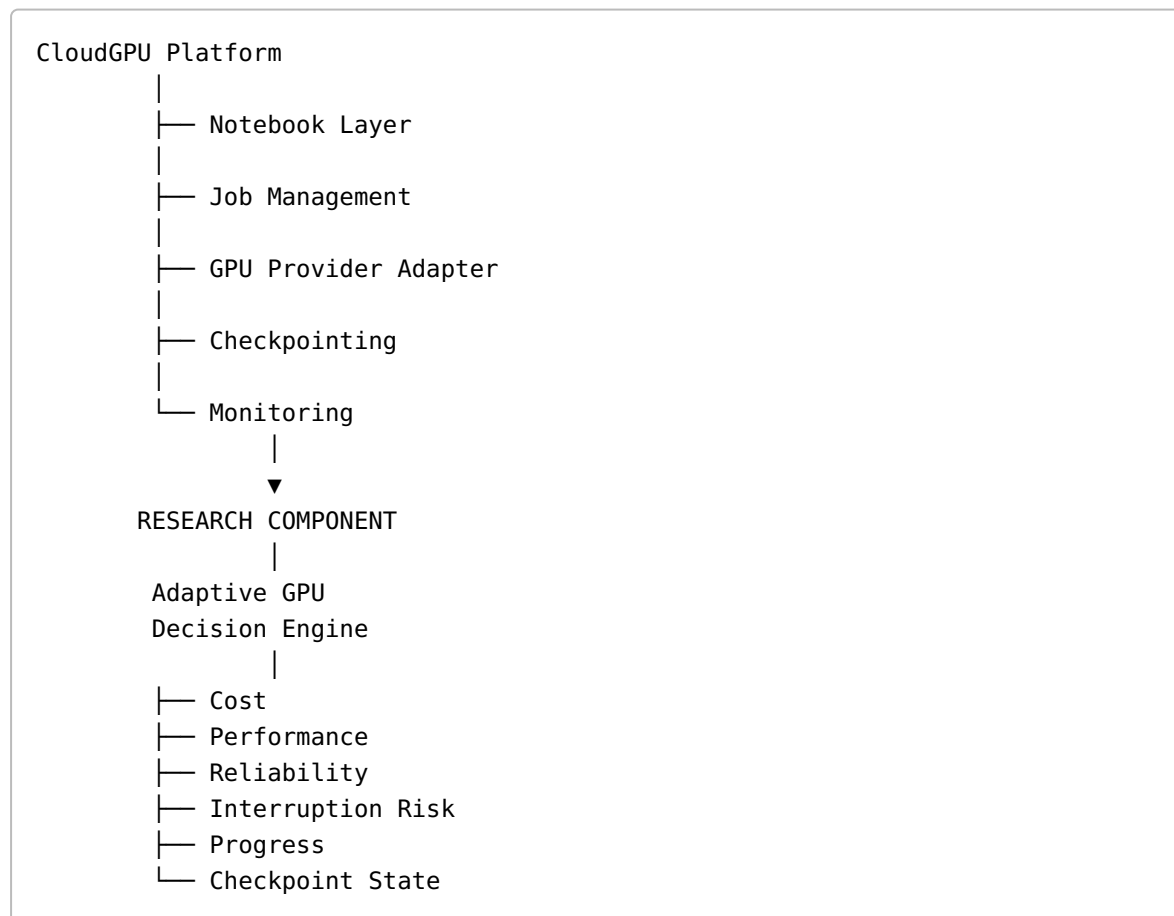
The existing CloudGPU Notebook Service will act as the **experimental platform**.

Its existing components include:

- Next.js;
- FastAPI;
- PostgreSQL;
- Redis;
- JupyterLab;
- JupyterHub;
- Docker;
- cloud GPU infrastructure;
- object storage;
- notification services.

These components are already specified in the project report.

The research layer will be added above the basic infrastructure:



This separation is important.

The platform itself is **engineering**.

The adaptive decision mechanism and its experimental evaluation constitute the **research**.

26. Security and Isolation

Because the platform executes user-provided machine-learning code, security is an important system limitation.

The current project report correctly identifies that user code should execute outside the API process and that Docker alone should not automatically be treated as a perfect hostile-code security boundary.

For the research prototype, the system should initially target trusted or controlled users.

Possible future isolation mechanisms include:

- dedicated VMs;
- microVMs;
- gVisor;
- Kata Containers;
- stronger cloud-provider isolation.

Security hardening will be treated as a system requirement rather than the primary research contribution.

27. Ethical Considerations

The research will primarily use controlled machine-learning workloads and infrastructure experiments.

Potential ethical considerations include:

- responsible use of cloud computing resources;
- avoiding unnecessary GPU consumption;
- protecting user credentials;
- protecting stored datasets and model artifacts;
- preventing unauthorized execution;
- avoiding exposure of cloud-provider credentials.

Cloud-provider API keys should remain server-side and should never be exposed to notebook code or browser applications. This is already identified in the system design.

28. Limitations

The study is expected to have several limitations.

1. Limited GPU availability

A university research project may not have access to a large heterogeneous GPU cluster.

2. Cloud pricing variability

Cloud GPU prices can change over time.

3. Provider dependency

Experiments using a single provider may not generalize to every cloud provider.

4. Limited workload diversity

A small number of ML workloads may not represent all machine-learning applications.

5. Security

A prototype may not provide production-grade isolation for arbitrary untrusted users.

6. Scheduling scale

A system tested with tens of jobs cannot automatically claim scalability to thousands of GPUs.

7. Model accuracy

The research does not directly optimize model accuracy.

29. Risk Management

Risk	Impact	Mitigation
GPU prices increase	High	Use controlled pricing traces/simulation
GPU unavailable	High	Provider abstraction and fallback resources
Training failure	High	Checkpointing
Provider API instability	Medium	Adapter + retries
Limited GPU budget	High	Small-scale experiments + simulation
Security vulnerability	High	Trusted users + isolation controls
Algorithm performs poorly	High	Multiple baselines and ablation studies
Insufficient novelty	High	Strong literature review and gap validation

30. Ablation Study

A strong paper should not only compare the complete proposed system against baselines.

It should also determine **which components actually matter**.

For example:

Proposed Full Model

Cost + Performance + Reliability + Interruption Risk + Checkpoint State

Ablation A

Remove reliability.

Ablation B

Remove checkpoint state.

Ablation C

Remove interruption risk.

Ablation D

Use static rather than dynamic decisions.

Ablation E

Use fixed checkpoint intervals.

This allows the research to answer:

Which factors actually contribute to the improvement?

That makes the paper substantially stronger.

31. Possible Research Algorithm

The eventual algorithm can follow this high-level process:

INPUT:

Training Job J

Candidate GPU Set G

1. Determine job requirements
↓
2. Retrieve available GPU offers
↓
3. Measure:
 - Cost
 - Performance
 - VRAM
 - Reliability
 - Interruption Risk
 - Startup Time↓
4. Retrieve:
 - Training Progress
 - Checkpoint State
 - Estimated Remaining Time↓
5. Calculate expected cost/risk
↓
6. Rank candidate GPUs
↓
7. Select optimal resource
↓
8. Start/resume training
↓
9. Continuously monitor:
 - Progress
 - Cost
 - GPU state
 - Reliability↓
10. Recalculate decision when necessary
↓
11. If failure occurs:
 - Retrieve latest checkpoint
 - Select replacement GPU
 - Resume training↓
12. Continue until completion

32. Evaluation Scenarios

The final research should evaluate at least these scenarios:

Scenario 1

Stable GPU market.

Scenario 2

Large price difference between GPU types.

Scenario 3

High interruption probability.

Scenario 4

Long-running training.

Scenario 5

Short-running training.

Scenario 6

Large checkpoint size.

Scenario 7

Multiple concurrent users.

Scenario 8

GPU failure near the beginning of training.

Scenario 9

GPU failure near the end of training.

The last two are particularly interesting because they test whether **training progress should influence resource decisions**.

33. Literature Review Structure

The literature review should be organized into themes rather than simply presenting 10 papers one after another.

Theme 1 — GPU Cluster Scheduling

- Gandiva
- Tiresias
- Optimus
- AntMan

Theme 2 — Adaptive and Goodput-Aware Scheduling

- Pollux
- Sia
- heterogeneous GPU scheduling research

Theme 3 — Cost-Efficient Cloud ML

- cost-aware cloud provisioning
- spot/preemptible GPU utilization
- cost-performance optimization

Theme 4 — Checkpointing and Failure Recovery

- checkpoint optimization
- preemption-aware training
- dynamic checkpoint intervals

Theme 5 — Notebook-Based ML Infrastructure

- Jupyter-based cloud platforms
- CloudX
- scalable notebook workflows

Theme 6 — Research Gap

Finally compare:

What has each paper optimized?

against:

What is the proposed research optimizing jointly?

34. Core Papers for the Literature Review

The initial literature set should include at least:

1. **Gandiva: Introspective Cluster Scheduling for Deep Learning** — OSDI 2018. [1]
2. **Optimus: An Efficient Dynamic Resource Scheduler for Deep Learning Clusters** — EuroSys 2018. [2]
3. **Tiresias: A GPU Cluster Manager for Distributed Deep Learning** — NSDI 2019. [3]
4. **AntMan: Dynamic Scaling on GPU Clusters for Deep Learning** — OSDI 2020. [4]

5. **Pollux: Co-adaptive Cluster Scheduling for Goodput-Optimized Deep Learning** — OSDI 2021. [5]
6. **A GPU Scheduling Framework to Accelerate Hyper-Parameter Optimization in Deep Learning Clusters (Hermes)** — Electronics 2021. [6]
7. **Sia: Heterogeneity-aware, Goodput-optimized ML-cluster Scheduling** — SOSP 2023. [7]
8. **CloudX: A Dual-Layered Cloud Native Architecture for GPU Resource Management Using Interactive Notebook** — Array 2026. [8]
9. **Cost-Efficient Training and Checkpointing for Large Models on Preemptible Cloud VMs** — EuroMLSys 2026. [9]
10. **Deep Learning Workload Scheduling in GPU Datacenters: Taxonomy, Challenges and Vision** — survey literature. [10]

The first seven establish that GPU scheduling, heterogeneity and adaptive resource allocation are already mature research areas. [1–7] CloudX and the 2026 checkpointing work are especially important because they represent very recent work that is close to the proposed system. [8][9]

35. Expected Research Output

The expected outputs are:

Software Output

A functional cloud GPU training orchestration prototype.

Algorithmic Output

An adaptive GPU resource-selection/orchestration mechanism.

Experimental Output

A controlled evaluation dataset containing:

- GPU characteristics;
- training workload information;
- resource decisions;
- costs;
- failures;
- checkpoints;
- recovery events;
- completion times.

Academic Output

A research paper demonstrating:

1. the research problem;
 2. literature gap;
 3. proposed approach;
 4. implementation;
 5. experimental methodology;
 6. results;
 7. comparison with baselines;
 8. limitations;
 9. conclusions.
-

36. Proposed Timeline

Phase 1 — Literature Review

Weeks 1–3

- Search literature
- Build literature matrix
- Identify precise gap
- Refine research question

Phase 2 — Research Design

Weeks 4–5

- Define optimization objectives
- Define algorithm
- Define baselines
- Define metrics

Phase 3 — Platform Development

Weeks 6–10

- Job orchestration
- GPU provider adapter
- checkpointing
- monitoring
- recovery

Phase 4 — Algorithm Implementation

Weeks 11–13

- Resource scoring
- adaptive decisions
- cost estimation
- failure-aware decisions

Phase 5 — Experiments

Weeks 14–17

- Baseline experiments
- failure experiments
- checkpoint experiments
- workload experiments
- multi-user experiments

Phase 6 — Analysis

Weeks 18–19

- Statistical analysis
- ablation studies
- visualization
- interpretation

Phase 7 — Paper

Weeks 20–22

- Results
- Discussion
- Conclusion
- Final paper

37. Proposed Paper Structure

The eventual research paper can follow:

1. Abstract

Problem → Gap → Method → Experiments → Results → Contribution

2. Introduction

Why cloud GPU training is expensive/unreliable.

3. Background

Cloud GPUs, persistent training, checkpointing and scheduling.

4. Related Work

GPU scheduling + cloud cost optimization + checkpointing + notebook platforms.

5. Research Gap

Precise distinction from existing approaches.

6. Proposed Approach

Adaptive orchestration model.

7. System Architecture

Control plane + compute plane + decision engine.

8. Algorithm

Formal model and algorithm.

9. Experimental Methodology

Workloads, GPU resources, baselines and failure scenarios.

10. Results

Cost, JCT, utilization, recovery and reliability.

11. Discussion

Why the approach worked or failed.

12. Limitations

Threats to validity and generalizability.

13. Conclusion

Research findings and future work.

38. Proposed Abstract — Initial Draft

This research proposes an adaptive cloud GPU orchestration approach for persistent machine learning training workloads. Long-running machine learning jobs executed on cloud GPU infrastructure face a trade-off between computational performance, resource cost and reliability, particularly when using heterogeneous or interruptible GPU resources. Existing GPU scheduling approaches have investigated dimensions including job completion time, GPU utilization, fairness, heterogeneity and adaptive resource allocation, while recent work has also explored cost-efficient training and checkpointing on preemptible cloud resources.

However, selecting an appropriate cloud GPU for a persistent training workload requires consideration of both resource characteristics and the evolving state of the training job. This research proposes an adaptive orchestration mechanism that jointly considers GPU cost, expected performance, reliability, interruption risk, training progress and checkpoint recovery state when making resource-selection decisions.

A prototype cloud-based persistent training platform will be developed to provide an experimental environment for long-running machine learning jobs. The system will decouple notebook interaction from background training execution and provide GPU provisioning, monitoring, checkpointing and recovery capabilities. The proposed approach will be evaluated against static and heuristic GPU-selection strategies under varying GPU prices, performance characteristics, interruption probabilities and workload conditions.

The evaluation will focus on total training cost, job completion time, GPU utilization, recovery time, computational loss, checkpoint overhead and successful job completion rate. The research aims to determine whether adaptive, checkpoint-aware GPU orchestration can provide a more effective cost-performance-reliability trade-off than conventional static resource-selection strategies.

39. Final Research Position

The proposed research should ultimately be positioned as:

Not a research project about building another Google Colab.

Instead:

A research project investigating adaptive orchestration of heterogeneous cloud GPU resources for persistent machine-learning training under cost, performance and reliability constraints.

The notebook platform provides the experimental environment.

The GPU orchestration algorithm provides the research contribution.

The experiments provide the evidence.

The comparison against existing scheduling and cost-aware approaches provides the academic justification.

References — Initial Literature Set

- [1] Xiao, W. et al. (2018). *Gandiva: Introspective Cluster Scheduling for Deep Learning*. OSDI 2018.
- [2] Peng, Y., Bao, Y., Chen, Y., Wu, C., & Guo, C. (2018). *Optimus: An Efficient Dynamic Resource Scheduler for Deep Learning Clusters*. EuroSys 2018.
- [3] Gu, J. et al. (2019). *Tiresias: A GPU Cluster Manager for Distributed Deep Learning*. NSDI 2019.
- [4] Xiao, W. et al. (2020). *AntMan: Dynamic Scaling on GPU Clusters for Deep Learning*. OSDI 2020.
- [5] Qiao, A. et al. (2021). *Pollux: Co-adaptive Cluster Scheduling for Goodput-Optimized Deep Learning*. OSDI 2021.
- [6] Son, J. et al. (2021). *A GPU Scheduling Framework to Accelerate Hyper-Parameter Optimization in Deep Learning Clusters*. Electronics, 10(3), 350.
- [7] Subramanya, S. J. et al. (2023). *Sia: Heterogeneity-aware, Goodput-optimized ML-cluster Scheduling*. SOSPP 2023.
- [8] Shiddiqi, A. M. et al. (2026). *CloudX: A Dual-Layered Cloud Native Architecture for GPU Resource Management Using Interactive Notebook*. Array, 31, 101026.
- [9] Desai, O., Pei, S., Bhimani, J., & Kim, B. S. (2026). *Cost-Efficient Training and Checkpointing for Large Models on Preemptible Cloud VMs*. Proceedings of the Sixth European Workshop on Machine Learning and Systems, 31–40.
- [10] *Deep Learning Workload Scheduling in GPU Datacenters: Taxonomy, Challenges and Vision* (2022). Survey of GPU scheduling research.